8 Ways to Protect Web Forms from Spam

Posted on: May 21, 2017 by Dimitar Ivanov

I'm sure that as site or blog owner you receive tons of spam comments. Besides being so annoying the spam comments can hurt your ranking in SERPs. Through the years are used different prevention techniques trying to stop the spam. This article aims to summarize most effective anti-spam methods.

1. Captcha
2. Honey pot
3. CSRF token
4. IP filter
5. Content filter
6. Referer header
7. Origin header
8. Hosted service

CAPTCHA

This is probably the most popular method **fighting the SPAM of web forms**. It works as following: a random text or more complex expression is generated and stored in a session, then the text is drawn over an image included in the form and visitors must input that text prior to submit the form.

```
<form action="comment.php" method="post">
    <p>
        <label>Human verification</label>
        <img src="captcha.php">
        <input type="text" name="captcha">
    </p>
</form>
```

Figure 1. Captcha

Afterward, the user input must be validated on the server side by comparison with a session value.

```
<?php
if ($_SESSION['captcha'] != $_POST['captcha']) {
    // It's SPAM
}
?>
```

This technique relies on assumption that this is an easy task for humans, unlike the computers. That was true in the past but the quick evolve of the OCR programs helps spammers to solve the captcha.

However, to build your own captcha you need to have some knowledge of a server-side language (PHP, Python, Ruby, etc.)

HONEY POT

This method relies on the assumption that SPAM software doesn't recognize CSS and/or JavaScript. The "**honey pot**" technique use a non-visible field to fool the less-intelligent robots whos automatically fills out all the input fields prior to submit the form data for further processing.

```
<form action="comment.php" method="post">
    <p>
        <label>Name</label>
        <input type="text" name="your_name">
    </p>
    <p>
        <label>Email</label>
        <input type="email" name="your_email">
    </p>
    <p class="fax">
        <label>Fax</label>
        <input type="text" name="your_fax">
    </p>
    <p>
        <label>Comment</label>
        <textarea name="your_comment"></textarea>
    </p>
    <p>
        <button type="submit">Submit</button>
    </p>
</form>
```

Then use CSS to hide the "honey pot" from your form so visitors are not able to see and fill it.

```
<style>
.fax {
    display: none;
}
</style>
```

You can also use JavaScript to assure yourself this input field will not harm your form.

```
<script>
document.querySelector('.fax').style.display = 'none';
</script>
```

So, if visitors can't see and fill the non-visible input fields we can consider that the form submission with not empty fax is spam.

```php
<?php
// comment.php
if (!empty($_POST['your_fax'])) {
    // It's SPAM
}
?>
```

CSRF TOKEN

Synchronizer token pattern uses a unique token that is embedded into the HTML forms and verified on the server side. The CSRF token should be a random value that is hard to predict, preferably generated by a cryptographical algorithm. This is **how to build a CSRF token**:

```php
<?php
// PHP 7
$token = bin2hex(random_bytes(32));

// PHP 5.3 with mcrypt
$token = bin2hex(mcrypt_create_iv(32, MCRYPT_DEV_URANDOM));

// PHP 5.3 with openssl
$token = bin2hex(openssl_random_pseudo_bytes(32));

// PHP 4
$token = base64_encode(time() . sha1($_SERVER['REMOTE_ADDR'] .
$_SERVER['HTTP_USER_AGENT']) . md5(uniqid(rand(), true)));

// Store the token into a session variable!
$_SESSION['token'] = $token;
?>
```

Then include the token into your HTML form.

```php
<form action="comment.php" method="post">
    <input type="hidden" name="token" value="<?php echo $token; ?>">
</form>
```

To validate a token you must compare the form value with the session value.

```php
<?php
// comment.php
if ($_SESSION['token'] !== $_POST['token']) {
    // It's SPAM
}
?>
```

IP FILTER

Create and regularly update a list with IP addresses from which you've received spam already. Then use the list to filter requests to your web forms. For more advanced and flexible IP filtering use regular expressions.

```php
<?php
$spammer_ip = array('178.137.136.76', '146.185.223.29');
if (in_array($_SERVER['REMOTE_ADDR'], $spammer_ip)) {
    // it's SPAM
}
?>
```

CONTENT FILTER

Create and regularly update a list with words considered as spam. Well-known topics include medicines, gambling, adult content, weight loss, etc. Then use regular expressions to find and block such a content.

```php
<?php
if (preg_match('/viagra|cialis|poker|casino/', $_POST['comment'])) {
    // it's SPAM
}
?>
```

REFERER HEADER

The `Referer` header shows the address of the page where a link to the current resource was followed. If your hostname is not present in its value the request is probably a spam.

```php
<?php
if (isset($_SERVER['HTTP_REFERER']) && strpos($_SERVER['HTTP_REFERER'],
'https://zinoui.com/') !== 0) {
    // it's SPAM
}
?>
```

ORIGIN HEADER

The `Origin` header shows where the request originates from. Its value includes only the scheme, server name, and the port number (only if the resource is served by a non-standard port). Unlike the *Referer* header, the *Origin header* does not include any path information. It is sent with CORS requests, as well as with POST requests. So, if the value of Origin header differs from your hostname the request is probably a spam.

```php
<?php
$allowed_origins = array('https://zinoui.com', 'https://www.example.org:8080');
if (isset($_SERVER['HTTP_ORIGIN']) && !in_array($_SERVER['HTTP_ORIGIN'],
$allowed_origins)) {
    // it's SPAM
}
?>
```

HOSTED SERVICE

A plenty number of third-party hosted anti-spam services are available. Most notable of them are Google reCaptcha and Akismet (Wordpress only).

CONCLUSION

Nowadays the spammers become more and more aggressive and the fight against it is a much difficult. That imposes to use as much as possible methods to **protect a web form from spam**. The fact that HTTP headers can be easily sent using the `cURL` or `XmlHttpRequest` should not discourage you to continue using them. They still have a place as an additional layer of defense to your programs.

SEE ALSO

- How to prevent SQL injections in PHP
- Best Web Performance Books
- Web App Install Banners

SHARE THIS POST

If you have a question about the *techniques for preventing SPAM*, please leave a comment below. And do not be shy to share this article. Thanks so much for reading!

Dimitar Ivanov

Dimitar Ivanov is a senior LAMP developer, javascript engineer, web performance-obsessed. He is programming since 2003 and loves to build web applications. You can find him on Twitter, LinkedIn and GitHub.

Subscribe to our newsletter

Join our mailing list and stay tuned! Never miss out news about Zino UI, new releases, or even blog post.

your@email.com Subscribe

0 Comments

Comments are closed

Zino UI **Products**